

Unit 2.Data Encapsulation and Inheritance

- 2.1 Access controls concepts (Public, Private, Protected) and differences among them.
 - 2.2 Declaring simple class, member variables and member functions.
 - 2.3 Concepts and use of enum.
 - 2.4 Concepts of Data hiding, abstraction and encapsulation with examples
 - 2.5 Concepts of Inheritance and Types of Inheritance
 - 2.6 Constructors and Destructors
-

2.1 Access controls concepts (Public, Private, Protected) and differences among them.

Access modifiers are used to implement an important aspect of Object-Oriented Programming known as data hiding. There are three types of access modifiers available in C++.

1. Private
2. Public
3. Protected

If we do not specify any access modifiers for the members inside the class, then by default the access modifier for the members will be private.

1. Public: All class members declared under the public specifier will be accessible to everyone. The data members and member functions declared as public can be accessed by other classes and functions as well. The public members of a class can be accessed from anywhere in the program using the direct member access operator (.) with the object of that class.

Example 2.1 Public Access Modifier

```
#include <iostream.h>
#include <conio.h>

class circle
{
    public:
    int r;
    void area()
    {
        cout<<"area "<<3.14*r*r;
    }
};

void main()
{
    clrscr();
    circle c;
    cout<<"enter radius";
```

```

        cin>>c.r;
        c.area();
        getch();
    }

```

Output enter radius 10
area 314

2. Private: The class members declared as private can be accessed only by the member functions inside the class. They cannot be accessed directly by any object or function outside the class. Only the member functions or friend functions are allowed to access the private data members of a class.

Example 2.2 Private Access Modifier

```

#include <iostream.h>
#include <conio.h>

class circle
{
    private:
    int r;
    public:
    void area()
    {
        cout<<"enter radius";
        cin>>r;
        cout<<"area "<<3.14*r*r;
    }
};

void main()
{
    clrscr();
    circle c;
    c.area();
    getch();
}

```

Output enter radius 10
area 314

3. Protected: The protected access modifier is similar to the private access modifier in that it can't be accessed outside of its class except with the help of a friend function. The difference is that the class members declared as protected can be accessed by any subclass (derived class) of that class as well.

Example 2.3 Protected Access Modifier

```
#include <iostream.h>
#include <conio.h>
class circle
{
    protected:
    int r;
    public:
    void area()
    {
        cout<<"enter radius";
        cin>>r;
        cout<<"area"<<3.14*r*r;
    }
};

void main()
{
    clrscr();
    circle c;
    c.area();
    getch();
}
```

Output enter radius 10
area 314

Example 2.4 Protected Access Modifier with Inheritance

```
#include <iostream.h>
#include <conio.h>
class radius
{
    protected:
    int r;
};

class circle:public radius
{
    public:
    void area()
    {
        cout<<"enter radius";
        cin>>r;
        cout<<"area "<<3.14*r*r;
    }
};
```

```
void main()
{
    clrscr();
    circle c;
    c.area();
    getch();
}
```

Output enter radius 10
area 314

2.2 Declaring simple class, member variables and member functions.

2.3 Concepts and use of enum.

An enumerated type (enumeration) is a user-defined data type that can be assigned a limited set of values. These values are defined by the programmer at the time of declaring the enumerated type.

It is also assigned starting from zero, similar to an array. Enumerations can also be used with switch statements.

Syntax: *enum enumerated-type-name{value1, value2, value3.....valueN};*

The enum keyword is used to declare enumerated types. After the enumerated type name is written, the possible values are defined within curly brackets. Once the enumerated type is defined, variables of that type can be created. There are two ways to create these variables.

1. They can be declared while declaring the enumerated type by adding the variable name before the semicolon.
2. Enumerated type variables can also be created later, just like normal variables.
enumerated-type-name variable-name = value; By default, the starting value of the first element of an enum is 0 (similar to arrays), but this can be changed explicitly.

Example 2.5 Enum example

```
#include <iostream.h>
#include <conio.h>

void main()
{
    // Defining enum Gender
    enum Gender { Male=1, Female }gen;

    // Creating Gender type variable
    Gender gender = Male;
    gen=Female;
    clrscr();

    cout<<gender<<endl;
    cout<<gen<<endl;
```

```

    getch();
}

```

```

Output: 1
        2

```

2.4 Concepts of Data hiding, abstraction and encapsulation with examples

Encapsulation is defined as wrapping up data and information into a single unit. In Object-Oriented Programming, encapsulation involves binding together the data and the functions that manipulate that data.

Consider a real-life example of encapsulation. In a company, there are different sections like the accounts section, finance section, sales section, etc. The finance section handles all financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all sales-related activities and keeps records of all sales.

Now, there may arise a situation where, for some reason, an official from the finance section needs all the data about sales for a particular month. In this case, he is not allowed to directly access the data of the sales section. He will first have to contact an officer in the sales section and then request the specific data. This is what encapsulation is. Here, the data of the sales section and the employees who can manipulate it are encapsulated under a single entity, the "sales section."

Data abstraction is one of the most essential and important features of object-oriented programming in C++. Abstraction means displaying only essential information and hiding the details. Data abstraction refers to providing only essential information about the data to the outside world while hiding the background details or implementation.

Consider a real-life example of a man driving a car. The man knows that pressing the accelerator will increase the speed of the car or applying the brakes will stop the car, but he does not know how pressing the accelerator actually increases the speed. He is unaware of the car's inner mechanisms or the implementation of the accelerator and brakes. This is what abstraction is.

2.5 Concepts of Inheritance and Types of Inheritance

The capability of a class to derive properties and characteristics from another class is called inheritance. Inheritance provides the concept of reusability. Classes can be reused through inheritance.

Subclass: The class that inherits properties from another class is called a subclass or derived class.

Superclass: The class whose properties are inherited by a subclass is called a base class or superclass.

```

Syntax: class subclass_name : access_mode base_class_name
{
    // body of subclass

```

```
};
```

The access modes are public, private, and protected. By default, the access mode is private.

Access mode

Public - This is the most commonly used inheritance mode. In this mode, the protected members of the superclass become protected members of the subclass, and the public members remain public.

```
class DerivedClassName : public BaseClassName
```

Private - In private mode, the protected and public members of the superclass become private members of the derived class.

```
class DerivedClassName : private BaseClassName
```

Protected - In protected mode, the public and protected members of the superclass become protected members of the subclass.

```
class DerivedClassName : protected BaseClassName
```

	Derived Class	Derived Class	Derived Class
Base Class	Public Mode	Private Mode	Protected Mode
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected
Public	Public	Private	Protected

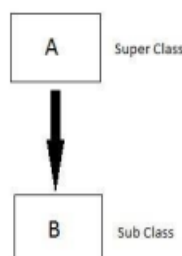
Table of Visibility Modes

Types of Inheritance in C++

1. Single Inheritance
2. Multiple Inheritance
3. Hierarchical Inheritance
4. Multilevel Inheritance
5. Hybrid Inheritance (Virtual Inheritance)

1. Single Inheritance

In this type of inheritance, one derived class inherits from only one base class. It is the simplest form of inheritance.



Example 2.6 Single Inheritance

```
#include <iostream.h>
#include <conio.h>
class A
{ public :
  int a;
  void getA()
  {cout<<"a"<<a<<endl; }
};
class B : public A
{ public :
  int b;
  void getB()
  { cout<<"b"<<b<<endl;}
};

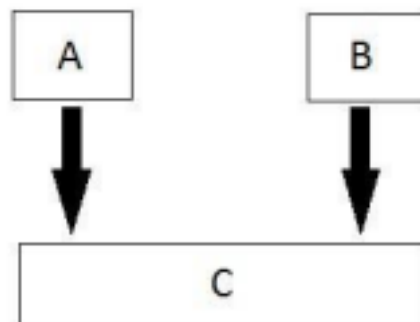
void main()
{ B b1;
  clrscr();
  cout<<"enter a";
  cin>>b1.a;
  cout<<"enter b";
  cin>>b1.b;
  b1.getA();
  b1.getB();
  getch();
}
```

Output

```
enter a1
enter b2
a1
b2
```

2. Multiple Inheritance

In this type of inheritance, a single derived class can inherit from two or more base classes.



Example 2.7 Multiple Inheritance

```

#include <iostream.h>
#include <conio.h>

class A
{
    public:
        int a;
        void geta()
        {
            cout<<"enter a";
            cin>>a;
        }
};

class B
{
    public:
        int b;
        void getb()
        {
            cout<<"enter b";
            cin>>b;
        }
};

class C:public A,public B
{
    public:
        void put()
        {
            cout<<"a "<<a<<" b "<<b;
        }
};

void main()
{
    C c;
    clrscr();
    c.geta();
    c.getb();
    c.put();
    getch();
}

```

Output

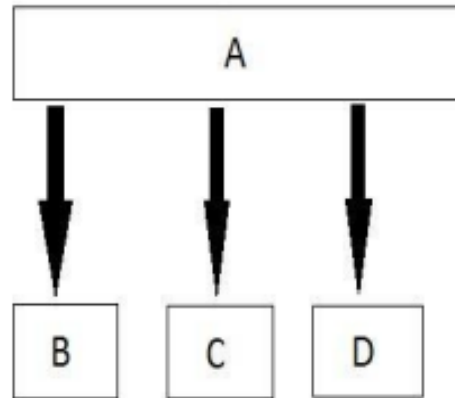
```

enter a1
enter b2
a 1 b 2

```


3.Hierarchical Inheritance

In this type of inheritance, multiple derived classes inherit from a single base class.



Example 2.8 Hierarchical Inheritance

```

#include <iostream.h>
#include <conio.h>

class A
{ public :
    int a;
    void getA()
    { cout<<" a "<<a<<endl; }
};

class B : public A
{ public :
    int b;
    void getB()
    { cout<<"b"<<b<<endl ; }
};

class C : public A
{ public :
    int c;
    void getC()
    { cout<<"c"<<c<<endl ; }
};

class D: public A
{ public :
    int d;
    void getD()
    { cout<<"d"<<d<<endl ; }
};

void main()
{ B b1;
  C c1;
  D d1;
  clrscr();
  cout<<"enter a";
  
```

```

cin>>b1.a;
cout<<"enter a";
cin>>c1.a;
cout<<"enter a";
cin>>d1.a;

cout<<"a from class b ";
b1.getA();
cout<<"a from class c ";
c1.getA();
cout<<"a from class d ";
d1.getA();

cout<<"enter b";
cin>>b1.b;
cout<<"enter c";
cin>>c1.c;
cout<<"enter d";
cin>>d1.d;

b1.getB() ;
c1.getC() ;
d1.getD() ;
getch();
}

```

Output :

```

enter a 1
enter a 2
enter a 3
a from class b a 1
a from class c a 2
a from class d a 3
enter b4
enter c5
enter d6
b4
c5
d6

```

4. Multilevel Inheritance

In this type of inheritance, the derived class inherits from a class which, in turn, inherits from another class. The superclass for one is a subclass for the other.



Example 2.9 Multilevel inheritance

```

#include <iostream.h>
#include <conio.h>

class A
{ public :
  int a;
  void getA()
  { cout<<"a"<<a<<endl ; } };

class B : public A
{public :
  int b;
  void getB()
  { cout<<"b"<<b<<endl ; } };

class C : public B
{ public :
  int c;
  void getC()
  { cout<<"c"<<c ; } };

void main()
{ C c1;
  clrscr();
  cout<<"enter a";
  cin>>c1.a;
  cout<<"enter b";
  cin>>c1.b;
  cout<<"enter c";
  cin>>c1.c;
  c1.getA();
  c1.getB();
  c1.getC();
  getch();
}

```

Output

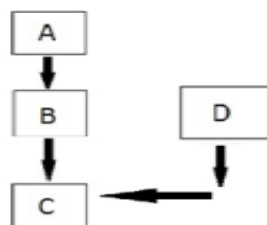
```

enter a 1
enter b 2
enter c 3
a1
b2
c3

```

5. Hybrid Inheritance

Hybrid inheritance is a combination of two or more types of inheritance.



Example 2.10 Hybrid Inheritance

```

#include <iostream.h>
#include <conio.h>

class A
{ public :
  int a;
  void getA()
  { cout<<"a"<<a<<endl; }
};

class D
{ public :
  int d;
  void getD()
  { cout<<"d"<<d<<endl; }
};

class B : public A
{ public :
  int b;
  void getB()
  { cout<<"b"<<b<<endl; }
};

class C : public B, public D
{ public :
  int c;
  void getC()
  { cout<<"c"<<c<<endl; }
};

void main()
{ C c1;
  clrscr();
  cout<<"enter a";
  cin>>c1.a;
  cout<<"enter b";
  cin>>c1.b;
  cout<<"enter c";
  cin>>c1.c;
  cout<<"enter d";
  cin>>c1.d;

  c1.getA();
  c1.getB();
  c1.getD();
  c1.getC();
  getch();
}

```

Output

```

enter a 1
enter b 2
enter c 3
enter d 4
a1
b2
d4
c3

```

Constructor with inheritance

Base class constructors are always invoked in the constructors of derived classes. Whenever you create a derived class object, the default constructor of the base class is executed first, followed by the completion of the derived class's constructor.

In order to invoke the parameterized constructor of the base class within the parameterized constructor of the derived class, it must be explicitly specified during the declaration of the derived class's parameterized constructor.

Example 2.11 Base class Constructor in Derived class Constructors

```

#include <iostream.h>
#include <conio.h>

class Base
{
    int x;
public:
    Base() // default constructor
    {
        cout << "Base default constructor\n";
    }

    Base(int i) // parameterized constructor
    { x=i;
      cout << "Base class parameterised constructor"<<x<<endl;
    }
};

class Derived : public Base
{
    int y;
public:
    Derived() // default constructor
    {
        cout << "Derived default constructor\n";
    }

    Derived(int i,int j):Base(j) // parameterized constructor

```

```

{ y=i;
cout << "Derived parameterized constructor"<<y<<endl;
}
};
void main()
{
clrscr();
Derived d1;
Derived d2(10,20);
getch();
}

```

Output

Base default constructor

Derived default constructor

Base class parameterised constructor 20

Derived parameterized constructor 10

Example 2.12 parameterised constructor in single inheritance

```
#include <iostream.h>
```

```
#include <conio.h>
```

```

class A
{
public:
int a;
A(int x)
{
a=x;
}
void puta()
{
cout<<a;
}
};
class B:public A
{
public:
int b;
B(int x,int y):A(y)
{
b=x;
}
void putb()
{
cout<<b;
}
}

```

```
};
void main()
{
    clrscr();
    B b1(1,2);
    b1.puta();
    b1.putb();
    getch();
}
```

Output

21

Example 2.13 parameterised constructor in multiple inheritance

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class A
```

```
{
    int a;
    public :
    A(int x)
    { a=x; }
    void putA()
    { cout<<a; }
};
```

```
class B
```

```
{
    int b;
    public :
    B(int x)
    {
        b=x;
    }
    void putB()
    { cout<<b; }
};
```

```
class C : public A ,public B
```

```
{
    int c;
    public :
    C(int x,int y,int z) : A(x),B(y)
    {
        c=z;
    }
    void putC()
    { cout<<c; }
};
```

```
};

int main()
{
    clrscr();
    C c1(1,2,3);
    c1.putA();
    c1.putB();
    c1.putC();
    getch();
    return(0);
}
```

Output
123

Example 2.14 parameterized constructor in multilevel inheritance

```
#include <iostream.h>
#include <conio.h>

class A
{
    int a;
public :
    A(int x)
    { a=x; }
    void putA() { cout<<a; }
};

class B : public A
{
    int b;
public :
    B(int x,int y):A(x)
    { b=y;}
    void putB() { cout<<b; }
};

class C : public B
{
    int c;
public :
    C(int x,int y,int z):B(x,y)
    { c=z; }
    void putC() { cout<<c; }
};

int main()
{
    clrscr();
    C c1(1,2,3);
    c1.putA();
```



```

    c1.putB();
    c1.putC();
    getch();
    return(0);
}

```

Output
123

2.6 Constructors and Destructors

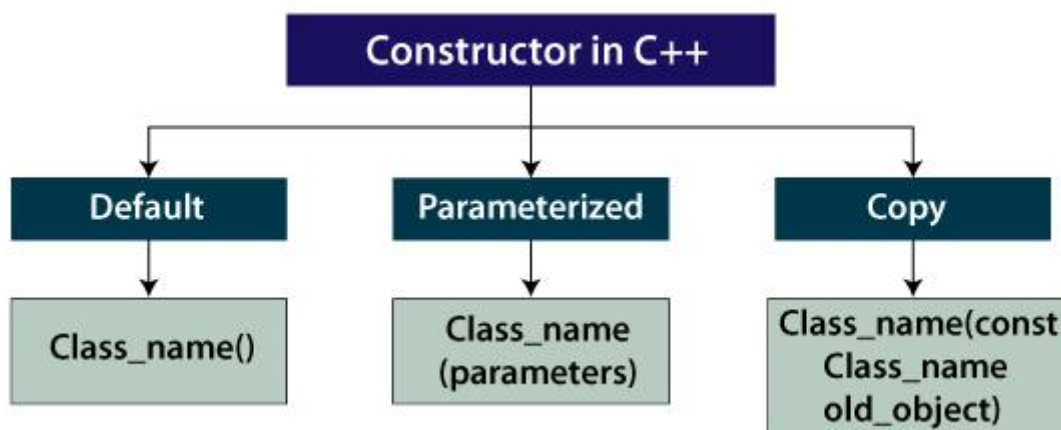
What is a constructor?

A constructor is a member function of a class that initializes objects of the class. In C++, a constructor is automatically called when an object (instance of the class) is created. It is a special member function of the class.

How are constructors different from a normal member function?

A constructor is different from a normal function in the following ways:

1. A constructor has the same name as the class itself.
2. Constructors don't have a return type.
3. A constructor is automatically called when an object is created.
4. If we do not specify a constructor, the C++ compiler generates a default constructor for us (which expects no parameters and has an empty body).



Default Constructors: A default constructor is a constructor that does not take any arguments and has no parameters.

Example 2.15 Default constructor

```

#include <stdio.h>
#include <iostream.h>
#include <conio.h>

```

```

class construct
{
    int a, b;
public:
    construct() // Default Constructor
    {
        a = 10;
        b = 20;
    }
    void putdata() { cout<<a<<b; }
};

int main()
{
    // Default constructor called automatically
    // when the object is created
    construct c;
    clrscr();
    cout << "value of a:b ";
    c.putdata() ;
    getch();
    return(0);
}

```

Output:
value of a:b1020

Parameterized Constructors: Parameterized Constructors allow for passing arguments to constructors, typically used to initialize an object when created. To create a parameterized constructor, add parameters to it and use these parameters in the constructor's body to initialize the object.

Example 2.16 Parameterized constructor

```

#include <iostream.h>
#include <conio.h>

class construct
{
    int a, b;
public:
    construct() { } // Default Constructor

    construct(int x,int y)//parameterized construcotor
    {
        a=x;
        b=y;
    }
}

```

```

void putdata() { cout<<a<<" "<<b; }
};

void main()
{
// when the object is created
clrscr();
construct c(5,10); //parameterized constructor
cout << "value of a:b ";
c.putdata() ;
getch();
}

```

Output:

value of a:b5 10

Uses of Parameterized constructor:

1. It is used to initialise the various data elements of different objects with different values when they are created.
2. It is used to overload constructors.

Copy Constructor: The copy constructor is a member function that initializes an object using another object of the same class.

Example 2.17 Copy constructor

```

#include <iostream.h>
#include <conio.h>

class Point
{
    private:
        int x, y;
    public:
        Point(int x1, int y1)//parameter constructor
        { x = x1; y = y1; }

        Point(Point &p) // Copy constructor
        { x = p.x;
          y = p.y;
        }

        void putdata() { cout<<x<<" "<<y<<endl; }
};

void main()
{
    Point p1(10, 15),p2(p1); /* Parameterized constructor is called here
    Point p2 = p1; Copy constructor is called here */
    clrscr();
}

```

```
p1.putdata();
p2.putdata();
getch();
}
```

Output

```
10 15
10 15
```

Multiple Constructor in a class

When more than one constructor function is defined in a class, we say that the constructor is overloaded.

Example of different types of constructors.

```
#include <iostream.h>
#include <conio.h>

class test
{
    public:
        int a,b;
        test()
        {
            cout<<"enter a and b";
            cin>>a>>b;
        }
        test(int x,int y)
        {
            a=x;
            b=y;
        }
        test(test &t)
        {
            a=t.a;
            b=t.b;
        }
        void display()
        {
            cout<<a<<b<<endl;
        }
};

void main()
{
    clrscr();
    test t,t1(11,22),t2(t1);
```

```

        clrscr();
        cout<<"\n Value of a and b with Default constructor";
        t.display();
        cout<<"\n Value of a and b with parameterized constructor";
        t1.display();
        cout<<"\n Value of a and b with copy constructor";
        t2.display();
        getch();
    }

```

Output:

enter a and b1 2

Value of a and b with Default constructor12

Value of a and b with parameterized constructor1122

Value of a and b with copy constructor1122

Default argument constructor

A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the calling function doesn't provide a value for the argument. In case any value is passed, the default value is overridden.

Example of default argument constructor

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class num
```

```
{
```

```
    public:
```

```
    int a,b;
```

```
    num(int x,int y=1) // constructor with default argument
```

```
    {
```

```
        a=x;
```

```
        b=y;
```

```
    }
```

```
    void display()
```

```
    {
```

```
        cout<<"\na "<<a<<"b "<<b;
```

```
    }
```

```
};
```

```
void main()
```

```
{
```

```
    clrscr();
```

```
    num n(10);
```

```
    num n1(10,20); // implicit call
```

```

num n2=num(4,5); //explicit call
n.display();
n1.display();
n2.display();
getch();
}
Output:
a 10b 1
a 10b 20
a 4b 5

```

What is a destructor?

A destructor is a member function that destructs or deletes an object. A destructor function is called automatically when the object goes out of scope, which can happen when the function ends, the program ends, a block containing local variables ends, or a delete operator is called.

How are destructors different from a normal member function?

Destructors have the same name as the class preceded by a tilde (~).

Example 2.18 Destructor

```

#include <iostream.h>
#include <conio.h>
int c=0;
class set
{
public:
set()
{
c++;
cout<<"\n construct "<<c;
}
~set()
{
cout<<"\n destruct "<<c;
c--;
}
};
void main()
{
clrscr();
{
set a,b;
}
getch();
}

```

Output
construct 1
construct 2
destruct 2
destruct 1